STATE 55 PAN-AFRICAN DIGITAL COMMUNITY

Complete System Architecture

Architecture Blueprint v1.2

Status: Production architecture baseline

Product direction: Continental, hybrid Pan-African digital community with membership, civic participation and member governance

Primary differentiator: African culture, heritage, knowledge, community discovery and member-led governance

Launch channels: Web (React) and mobile (Flutter)

Engineering objective: Architect, deploy and operate a secure, scalable continental platform using React, Flutter, Go, PostgreSQL, Redis, MinIO and SMTP, with automated CI/CD, observability, and Coolify-managed servers held to production operational standards.

1. EXECUTIVE SUMMARY

The State 55 Pan-African Digital Community is a digital platform connecting people across Africa and the African diaspora through culture, heritage, knowledge, communities, membership, conversations, creators, civic participation and member-led governance.

The platform replicates and extends an existing social application model: organizations and membership, digital membership cards, unique member IDs, volunteer activity tracking, leader evaluation, and online elections for virtual offices. These are built on top of the cultural and community features defined in v1.1. Payments, rewards, meeting rooms, diaspora services and the virtual passport follow in Phase 2. The wallet, investment exchange and trade exchange follow in Phase 3.

The recommended initial architecture is a Go modular monolith deployed as containers managed by Coolify on servers controlled by State 55. The web application uses React 19, Vite 7, Tailwind CSS, Radix UI and react-router 7. The mobile application uses Flutter. Both clients use a versioned REST API documented with OpenAPI. PostgreSQL, accessed through GORM with versioned SQL migrations, is the primary transactional database. Redis supports caching and ephemeral workloads, MinIO holds media and documents, SMTP delivers email, and asynchronous workers handle notifications, media processing, card generation and search indexing.

Production application containers run behind the Coolify proxy with health checks, rolling updates and measured horizontal scaling. Data services run on a separate private server with scheduled backups copied off-site. Availability is limited by the number of servers, and the HA and DR section states the limits and the mitigations plainly.

Architectural Principles

- African-first product context without treating Africa as culturally homogeneous.
- Inclusive, community-oriented governance with transparent and auditable decision processes.
- Privacy and security by design, with member affiliation and voting data treated as sensitive.
- Modular application boundaries before microservices.
- PostgreSQL as the source of truth for transactional data, with GORM models owned by one module each.
- Media and documents stored outside the relational database, in MinIO.
- API contracts documented with OpenAPI.
- Deployment and infrastructure configuration version-controlled.
- Observability and recovery designed from the beginning.
- Scale based on measurements rather than assumptions.
- Votes, balances and audit records stored as append-only, verifiable data.

2. PRODUCT SCOPE

2.1 Core product modules

Module	Responsibilities
Identity	Registration, authentication, sessions, verification, MFA, unique member ID
Profiles	User profiles, country, languages, constituency, diaspora status, interests, preferences
Organizations	Member organizations, wards, constituencies, hierarchy, organization roles
Membership	Applications, status, expiry, digital and printable membership card, public card verification
Social	Posts, comments, reactions, follows, feeds
Communities	Community creation, membership, roles, group discussions, local problem resolution
Culture & Heritage	Cultural stories, heritage records, attribution, cultural categories
Africa Atlas	Country and cultural discovery, geographic navigation
Discovery	Search, recommendations, trending, exploration
Creators	Creator profiles, portfolios, published work
Events	Events, attendance, registration, community activities
Documents	Downloadable forms, certificates and organization documents
Volunteering	Volunteer activities, hours, attendance, recognition
Governance	Virtual offices, candidate nominations, adoption certificates per constituency, elections, ballots, results, leader terms
Evaluations	Leader performance ratings and reviews
Notifications	In-app, email and push notification orchestration
Moderation	Reports, reviews, enforcement, appeals
Administration	Platform configuration, operations, administrative controls

2.2 Phase 1 scope (MVP)

Phase 1 should include:

- Registration, login, MFA and profile management, with a unique member ID for every member.
- Organizations and membership, with the digital membership card, printable PDF and public verification.
- Posts, comments, reactions and follows.
- Community creation and membership.
- Cultural stories and country pages.
- Documents and downloads.
- Volunteer activity tracking.
- Leader evaluation ratings.
- Governance: virtual offices for president, cabinet and parliament, candidate nomination, adoption certificates per constituency, online elections and auditable results.
- Search and basic discovery.
- Image and media references.
- Notifications over in-app, SMTP email and push.

- Reporting and moderation.
- Administrative console.

2.3 Features deferred from Phase 1

Phase 2

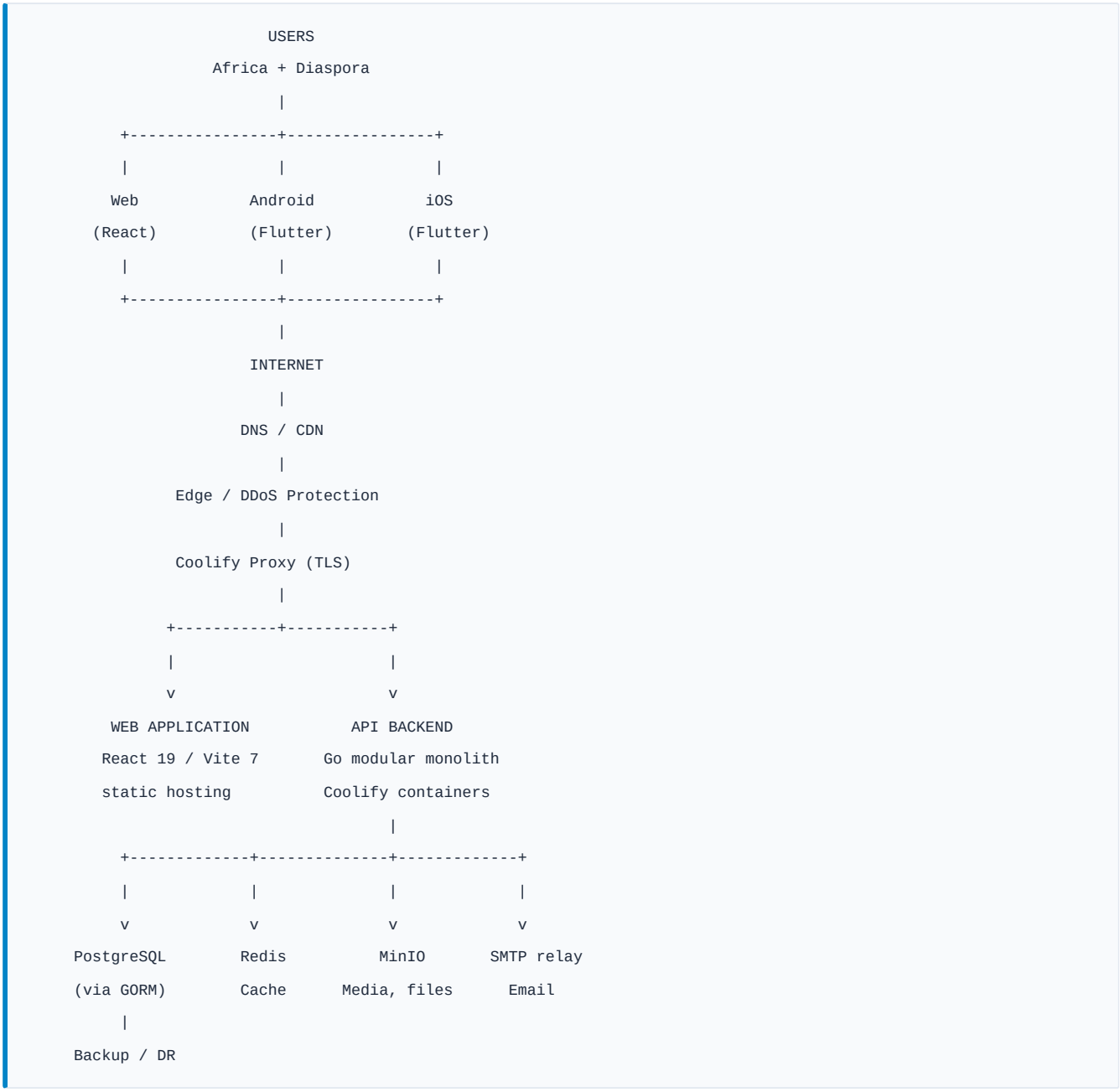
- Payments: membership fees, subscriptions and voluntary contributions through mobile money and bank cards.
- Subscriptions and paid member access.
- Rewards for activity and contributions.
- Virtual meeting rooms using an embedded hosted provider.
- Diaspora services: OISAM card application workflow and diaspora directory.
- Virtual passport, issued as a membership credential and not as a legal travel document.

Phase 3

- Wallet and virtual currency.
- Investment exchange for crowdfunded mines and energy projects, with project validation.
- Trade exchange and vending for commodities, tourism and health services.
- Livestreaming infrastructure and large-scale video hosting and transcoding.
- Advanced recommendation models.
- Cryptocurrency integrations.
- Multi-region active-active application deployment.

Outside this architecture: the African search engine and the proprietary mobile operating system are separate programmes of work and are not designed here.

3. HIGH-LEVEL SYSTEM ARCHITECTURE



This is a logical architecture. PostgreSQL, Redis and MinIO run as Coolify-managed resources on a private data server. Managed equivalents can replace them through configuration only.

4. APPLICATION ARCHITECTURE

4.1 Backend pattern

Use a Go modular monolith initially. Modules are separated by business responsibility and internal package contracts, but deployed as one API application with a separately invokable worker process where needed. The binary exposes `serve`, `worker`, `migrate` and `seed` commands.

Avoid premature microservices. Separate services only when scaling, isolation, deployment independence or organizational requirements justify the operational cost.

4.2 Backend module boundaries

Module	Core responsibilities
Identity	Registration, login, verification, sessions, MFA, member ID issuance
Profiles	User identity, country, language, constituency, diaspora status, interests
Organizations	Organization hierarchy, wards, constituencies, organization roles
Membership	Applications, status, expiry, card issuance, revocation and verification
Social	Feed, posts, comments, reactions, follows
Communities	Community creation, membership, roles
Culture	Heritage records, cultural stories, attribution
Discovery	Search, recommendations, trending
Creators	Creator identity, portfolio, published content
Events	Event creation, attendance, registrations
Documents	Document catalogue, visibility rules, downloads
Volunteering	Activities, hours, attendance, recognition
Governance	Offices, nominations, adoption certificates, elections, ballots, results
Evaluations	Leader ratings, reviews, aggregation
Notifications	Preferences, notification creation and delivery
Moderation	Reports, reviews, actions, appeals
Administration	Platform configuration and privileged operations
Payments, Subscriptions, Rewards, Meetings, Diaspora, Passport	Phase 2 modules. Package and schema reserved, no logic until Phase 2
Wallet, Exchange, Vending	Phase 3 modules. Package and schema reserved, no logic until Phase 3

4.3 Module design rules

- Each module owns its business logic, models and data access.
- Other modules use published interfaces rather than internal repositories.
- Cross-module database access is strictly controlled. No GORM associations or `Preload` across module boundaries.
- Domain events support asynchronous workflows.
- Shared utilities must not become a dumping ground for business logic.
- Module dependencies should be documented and tested.
- Go packages and module tests document and enforce module boundaries.

4.4 Data access with GORM

- GORM is the data access layer. Each module has its own models and repository, and one PostgreSQL schema.
- Production schema changes are versioned, reviewed SQL migrations. `AutoMigrate` is permitted in local development only.

- Connection pool limits (maximum open, maximum idle, connection lifetime) are set on every instance.
- Money is stored as `numeric(18,2)` or integer minor units and handled with a decimal type. Floating-point types are not permitted for money.
- Writes that must publish an event also write a transactional outbox row in the same transaction.

4.5 Background processing

A worker process runs asynchronous jobs through Asynq on Redis: email delivery, push notifications, card generation, image processing and search indexing. Scheduled jobs take a PostgreSQL advisory lock so that multiple replicas never run the same job twice. Workers expose the same liveness and readiness endpoints as the API.

5. API ARCHITECTURE

5.1 API entry point

All clients access the backend through a public API domain, for example: `https://api.example.africa/v1`. Web and mobile clients must not connect directly to PostgreSQL or Redis. They reach MinIO only through short-lived presigned URLs issued by the API.

5.2 API route structure

Endpoint	Module
`/v1/auth/*`	Identity
`/v1/users/*`	Profiles
`/v1/organizations/*`	Organizations
`/v1/memberships/*`, `/v1/cards/*`	Membership
`/v1/feed/*`, `/v1/posts/*`	Social
`/v1/communities/*`	Communities
`/v1/culture/*`	Culture
`/v1/atlas/*`	Culture / Discovery
`/v1/search/*`	Discovery
`/v1/creators/*`	Creators
`/v1/events/*`	Events
`/v1/documents/*`	Documents
`/v1/volunteering/*`	Volunteering
`/v1/governance/*`	Governance
`/v1/evaluations/*`	Evaluations
`/v1/notifications/*`	Notifications
`/v1/reports/*`	Moderation
`/v1/admin/*`	Administration

5.3 Example API operations

```
POST   /v1/auth/register
POST   /v1/auth/login
POST   /v1/auth/logout
GET    /v1/users/me

GET    /v1/memberships/me/card
GET    /v1/cards/verify/{cardCode}           (public)
POST   /v1/organizations/{id}/members

GET    /v1/feed
POST   /v1/posts
POST   /v1/posts/{postId}/comments
POST   /v1/posts/{postId}/reactions

GET    /v1/communities
POST   /v1/communities/{id}/join

GET    /v1/culture/stories
GET    /v1/culture/countries/{code}
GET    /v1/documents
POST   /v1/volunteering/activities
POST   /v1/evaluations

GET    /v1/governance/elections
POST   /v1/governance/elections/{id}/nominations
POST   /v1/governance/elections/{id}/ballots
GET    /v1/governance/elections/{id}/results

GET    /v1/search?q=african+heritage
POST   /v1/events
POST   /v1/reports
GET    /v1/notifications
```

5.4 API standards

The platform implements REST APIs using JSON, OpenAPI documentation, explicit versioning, request validation, consistent error responses, robust authentication and authorization, rate limiting, idempotency for sensitive operations such as registration and ballot casting, correlation IDs, structured request logs, distributed tracing, and clear deprecation policies.

Typed API clients for TypeScript and Dart are generated from the OpenAPI document in CI. A contract change that breaks a client fails the build.

Pagination Standard: Cursor pagination is preferred for feeds (for example `GET /v1/feed?limit=20&cursor=abc123`) because offset-based queries degrade severely as dataset volumes expand.

6. DATABASE ARCHITECTURE

6.1 Primary database

PostgreSQL 16 or a currently supported stable version is the primary transactional database, accessed through GORM. Start with one logical database and organize ownership by module through schemas. Avoid database-per-module complexity in Phase 1.

6.2 Core entity relationships

```
USERS            ||--||   USER_PROFILES         (has)
USERS            ||--o{   MEMBERSHIPS           (holds)
ORGANIZATIONS    ||--o{   MEMBERSHIPS           (contains)
ORGANIZATIONS    ||--o{   ORGANIZATIONS         (parent of)
MEMBERSHIPS      ||--o{   MEMBERSHIP_CARDS      (issued)
USERS            ||--o{   POSTS                 (creates)
USERS            ||--o{   COMMENTS              (writes)
USERS            ||--o{   COMMUNITY_MEMBERSHIPS (joins)
COMMUNITIES      ||--o{   POSTS                 (hosts)
POSTS            ||--o{   COMMENTS              (contains)
USERS            ||--o{   FOLLOWS               (follows)
USERS            ||--o{   VOLUNTEER_ACTIVITIES  (logs)
USERS            ||--o{   EVALUATIONS           (submits)
OFFICES          ||--o{   ELECTIONS             (contested in)
ELECTIONS        ||--o{   CANDIDACIES           (has)
ELECTIONS        ||--o{   BALLOT_RECEIPTS       (records voters)
ELECTIONS        ||--o{   BALLOTS               (holds votes)
CONSTITUENCIES   ||--o{   ADOPTION_CERTIFICATES (issues)
COUNTRIES        ||--o{   CULTURAL_STORIES      (associated)
EVENTS           ||--o{   EVENT_ATTENDEES       (includes)
USERS            ||--o{   NOTIFICATIONS         (receives)
USERS            ||--o{   CONTENT_REPORTS       (submits)
```

6.3 Initial table inventory

Domain	Tables
Identity	`users`, `credentials`, `user_sessions`, `user_mfa`, `member_ids`
Profiles	`user_profiles`, `user_languages`, `user_interests`
Organizations	`organizations`, `organization_roles`, `constituencies`, `wards`
Membership	`memberships`, `membership_cards`, `card_events`
Social	`posts`, `post_media`, `comments`, `reactions`, `follows`
Communities	`communities`, `community_memberships`, `community_roles`
Culture	`countries`, `languages`, `cultural_stories`, `cultural_tags`
Discovery	`content_tags`, `search_documents`
Creators	`creator_profiles`, `creator_portfolios`
Events	`events`, `event_attendees`

Domain	Tables
Documents	`documents`, `document_versions`, `document_access`
Volunteering	`volunteer_activities`, `volunteer_hours`
Governance	`offices`, `office_terms`, `elections`, `candidacies`, `adoption_certificates`, `ballot_receipts`, `ballots`, `election_results`
Evaluations	`evaluations`, `evaluation_periods`
Notifications	`notifications`, `notification_preferences`, `email_log`
Moderation	`content_reports`, `moderation_actions`, `appeals`
Administration	`admin_roles`, `admin_permissions`, `audit_events`

6.4 Database design rules

- UUID identifiers for public-facing entities; UTC timestamps internally.
- Standardized country identifiers; explicit content ownership and visibility.
- Foreign keys, appropriate indexes, and soft deletion where justified.
- Immutable audit records for critical actions, written in the same transaction as the action.
- Schema changes are versioned SQL migrations run by the `migrate` command. GORM `AutoMigrate` is used in local development only.
- No media binaries stored directly in PostgreSQL.
- Connection pooling with explicit limits; least-privilege database roles per environment.
- Encryption in transit and at rest where supported.
- Money values use decimal types, never floating point.
- Ballot content is stored separately from the record of who voted, so the two cannot be joined.

PostgreSQL remains the source of truth. Search indexes and caches are derived data and must be rebuildable.

6.5 Example data workflow



Use a transactional outbox pattern where reliable event publication is required.

7. INFRASTRUCTURE ARCHITECTURE

7.1 Environments

Environment	Purpose
Development	Local coding and debugging
Staging	Integration testing and deployment validation
Production	Real users and live application

Do not casually copy production data into development or staging. Use synthetic or appropriately anonymized test data. Staging email goes to a catch-all inbox, never to real members.

7.2 Local development architecture

```
Developer Workstation
  |
  +-- VS Code
  +-- Git
  +-- Go toolchain
  +-- Node.js (React web) and Flutter SDK (mobile)
  +-- Native Go API and worker processes
  +-- Local or managed PostgreSQL
  +-- Local Redis
  +-- Local MinIO
  +-- Local SMTP catcher for email testing
```

Local services are installed natively or started as containers. The Go API and workers run directly as native processes. GORM `AutoMigrate` is permitted locally only; all shared environments use the `migrate` command.

7.3 Application process management

- Build reproducible, versioned container images from Go binaries, tagged with the commit SHA and never `latest`.
- Pin base image versions, run as a non-root user, and define a container healthcheck.
- Configure environment-specific settings through Coolify environment variables and its secret store.
- Expose liveness and readiness endpoints. Readiness returns 503 when PostgreSQL or Redis is unreachable.
- Set container resource limits and restart policies.
- Write structured application logs to standard output for central collection.
- Pin and scan dependencies; generate a software bill of materials where practical.
- Refuse to start when a required secret is missing or equals a known default.

7.4 Coolify deployment

Coolify manages the application containers on servers controlled by State 55. The production baseline is one application server and one separate data server. Additional application servers are added through Coolify as measured load requires.

The hosting platform and surrounding automation must provide:

- Git-based deployments from GitHub with a documented rollback path.
- Reverse proxy with automatic TLS certificates.
- Rolling updates gated by container healthchecks.
- Environment variables and secrets per environment.
- Persistent volumes for PostgreSQL, Redis and MinIO.
- Scheduled database backups to S3-compatible storage.
- Server resource monitoring and alerting.
- Server firewall rules, DNS records and server provisioning kept as version-controlled scripts or infrastructure as code.

8. COMPUTE, NETWORK AND DEPLOYMENT ARCHITECTURE

8.1 Production compute structure

A production environment should include:

- An application server running the web, API and worker containers.
- A separate data server running PostgreSQL, Redis and MinIO on a private network.
- At least two API instances behind the Coolify proxy with health checks.
- Dedicated API and worker containers, scaled independently when needed.
- Restricted administrative access by SSH key only, with password login disabled.
- Central monitoring, log collection and host-level health reporting.
- Off-site backup storage with a different provider from the servers.

8.2 Logical service groups

Service group	Workloads
Public edge	DNS, CDN, DDoS protection, Coolify proxy
Web	React assets served as static files
Application	Go API and background worker containers
Data	PostgreSQL, Redis and MinIO on the private data server
Observability	Metrics, logs, traces and alerting
Administration	Restricted operator, SSH and Coolify dashboard access

Network boundaries and identity-based access policies provide isolation; logical service groups alone are not a security boundary.

8.3 Workload mapping

Workload	Hosting component
API backend	Go container on the application server, managed by Coolify
Background workers	Go worker container, deployed as a separate Coolify application
Web frontend	Static container behind the Coolify proxy, optionally fronted by a CDN
External routing	Coolify proxy with TLS termination
Database	PostgreSQL on the data server
Cache and queues	Redis on the data server
File storage	MinIO on the data server
Email	External SMTP provider
Configuration and credentials	Coolify environment variables and secret store
Scaling	Additional replicas and, later, additional application servers
Backups	Scheduled dumps to MinIO, copied to off-site storage

8.4 Initial replica baseline

Component	Staging baseline	Production baseline
Web	1	1 container behind a CDN
API	1	2+ instances
Worker	1	1+ instances
PostgreSQL	1	1 primary with scheduled backups; standby added when budget allows
Redis	1	1
MinIO	1	1, mirrored off-site

Production instance counts scale based on measured CPU and memory utilization, request load, queue lag and verified traffic volume. Validate capacity and scaling behavior through load tests before increasing limits.

9. NETWORKING AND LOAD BALANCING

9.1 Traffic architecture

```
Public Internet
      |
      v
DNS / CDN / Edge Protection
      |
      v
Coolify Proxy (TLS termination)
      |
      v
Private Application Network
      |
      +----> Go API containers
      +----> Go worker containers (internal only)
      |
      v
Private Data Network
      |
      +----> PostgreSQL
      +----> Redis
      +----> MinIO
```

9.2 Component responsibilities

Component	Responsibility
DNS	Resolve domain names
CDN	Cache eligible static assets and public media
Coolify proxy	Terminate TLS, route HTTP requests to healthy containers, apply edge policies
Container runtime	Start, monitor and restart application containers
Scaling procedure	Add replicas or servers based on measured load

9.3 Routing example

```
app.example.africa
    |
    +----> Web frontend

api.example.africa
    |
    +----> API containers
              +---- /v1/posts
              +---- /v1/cards
              +---- /v1/governance
```

The Coolify proxy handles routing, TLS termination, rate-limiting hooks and controlled traffic shifts between application versions.

9.4 Scaling

Coolify does not scale capacity automatically from metrics. Scaling is therefore a reviewed operational action, triggered by alerts on CPU and memory thresholds, request rate, latency and worker queue lag. The proxy routes traffic strictly to containers that pass their health checks.

9.5 Traffic management and production resilience verification

1. Verify traffic distribution across multiple API instances under sustained load.
2. Validate health-based routing and zero-downtime rolling deployments.
3. Simulate container failure and test graceful termination (SIGTERM handling and connection draining).
4. Validate scaling thresholds under stress and worker queue backlog.
5. Execute health-check failure and automatic container restart tests.
6. Simulate loss of the application server and verify the recovery runbook meets the agreed RTO.
7. Enforce TLS termination, HTTP/2 or HTTP/3 support and rate-limiting policies at the public edge.
8. Validate connection pooling, circuit breakers and backend timeout protections.
9. Measure and enforce latency (p95/p99), throughput and error budget (SLO) baselines.
10. Perform canary release validation where the deployment process supports it.

10. SECURITY ARCHITECTURE

Security is layered across edge, network, application, workload, data and operations.

10.1 Security zones

Zone	Access Policy
Public edge	Internet-accessible HTTPS
Application	Authorized internal routing
Data	Private access from authorized workloads
Administration	Restricted operator access
Observability	Authorized monitoring access
CI/CD	Controlled deployment permissions

10.2 Security boundaries

- PostgreSQL and Redis must never be publicly exposed.
- MinIO is reachable from the internet only through the proxy, for public media and presigned URLs. Its administration console stays private.
- Server administration is restricted to approved operator paths: SSH keys only, password login disabled, firewall open on ports 80 and 443 plus restricted SSH.
- Administrative endpoints must be protected, and containers run as unprivileged users.
- Production secrets must never be committed to source control, including example and template files. Compiled binaries are not tracked in Git.
- Network and host firewall rules must restrict unnecessary service-to-service traffic.
- Privileged access must be systematically audited; management interfaces are restricted from public access.

10.3 Identity and access security

Implement secure registration and login, email verification, session management, MFA for administrators and for any role that manages elections, organizations or money, Argon2id password hashing, role-based authorization through a declarative policy table, organization-level and community-level authorization, privileged action audit logs, session revocation and robust account recovery controls.

- Browser clients use httpOnly, secure, same-site cookie sessions. Tokens are not stored in `localStorage`.
- Mobile clients keep tokens in the platform secure store.
- Access and refresh tokens carry different type claims, and the API rejects a refresh token presented as an access token.
- The proxy removes client-supplied identity headers. Identity comes only from the verified session.
- Rate limiting trusts forwarded client addresses only from the Coolify proxy, and login limits fail closed when Redis is unavailable.

10.4 Member ID and membership card

Member ID. Each person has an internal UUID and a separate public member ID made of a fixed prefix, random characters and a check digit. The public ID is never sequential and reveals nothing about country or join date. One person holds one ID, enforced through verified email or phone, and through document checks for roles that require them.

Membership card. The card is a signed credential.

- The QR code carries the card ID, member ID, expiry and an ES256 signature. The signing key is held in the secret store, and a key ID in the card supports rotation.
- A public verification endpoint returns only valid, expired or revoked, plus the member's name and organization.

- Cards can be revoked immediately. The mobile application refreshes its offline copy whenever it is online.
- The card image and printable PDF are generated by a worker and stored in the private `cards` bucket.

10.5 Governance and election security

Governance is part of Phase 1, so the following controls are part of the Phase 1 baseline.

- Eligibility is enforced on the server from membership status, organization and constituency. Each eligible member can cast one ballot per election, enforced by a database constraint.
- Ballot secrecy: ballot content and the record of who voted live in separate tables with no join key. A receipt confirms participation only.
- Integrity: ballots are append-only. Results are computed by a reproducible job, and a hash-chained election log supports independent verification.
- Election configuration is frozen when voting opens. Opening, closing and certifying an election require two administrators with MFA, and every state change is written to the audit log.
- Adoption certificates are issued once per constituency, enforced by a unique constraint, to prevent duplicate candidates.
- Each election carries a mode flag, symbolic or binding. Binding mode stays disabled until an independent audit and legal review is complete.
- Ballot and nomination endpoints have strict rate limits, idempotency keys and anomaly alerts for duplicate accounts, impersonation and coordinated manipulation.

10.6 Infrastructure and access security

Control	Purpose
IAM / RBAC	Restrict Coolify, GitHub and server permissions
Network controls	Limit service communication and expose only required ports
Host hardening	Reduce operating-system attack surface, automatic security updates
Secrets management	Protect credentials in the Coolify secret store
Dependency scanning	Detect vulnerable Go, Dart and JavaScript dependencies
Deployment policy	Enforce approved image and release controls
Resource limits	Prevent resource exhaustion
Audit logging	Record administrative actions

10.7 Application process security

- Build Go binaries from trusted, pinned dependencies.
- Run containers as dedicated unprivileged users.
- Keep secrets outside images and source control.
- Dependency pinning and updates; source and dependency vulnerability scanning.
- Generate a Software Bill of Materials where practical.
- Sign release images and verify them before deployment as the platform matures.

10.8 Application and API security

Design against relevant OWASP risks: broken access control, authentication failures, injection, security misconfigurations, resource exhaustion, sensitive data exposure, API abuse, SSRF and improper input validation.

Social-platform-specific protections must also address impersonation, spam, harassment, fraudulent accounts, malicious links and coordinated manipulation.

Member affiliation, voting participation and political opinion are sensitive personal data. The platform applies data minimization, explicit consent, purpose limits and no sharing of member lists, and completes a data protection review before launch. Diaspora members live under the data protection laws of other countries, which the review must cover.

10.9 Policy and legal decisions that affect the design

Topic	Why it matters	Design response
Online elections	Ballot secrecy, voter identity, coercion and disputed results	Symbolic mode first, binding mode only after independent audit and legal advice
Virtual passport (Phase 2)	The name suggests a government travel document	Present it as a membership credential, with wording that it is not a legal identity or travel document
Money and virtual currency (Phases 2 and 3)	Payment, e-money and fundraising rules apply, and the platform becomes a fraud target	Licensed aggregator and hosted checkout first, decimal double-entry ledger, reconciliation, legal review before any wallet
Investment exchange (Phase 3)	Offering investments in mines and energy can fall under securities rules	Legal opinion first, with project validation performed by named people rather than by the application
Ownership and neutrality	v1.1 required the platform not to be controlled by a single party, while governance features imply a movement-led platform	Decide who owns the data, the brand and the production keys before launch

These points are flagged for review by a qualified lawyer and are not settled by this document.

11. OBSERVABILITY ARCHITECTURE

11.1 Monitoring stack

Technology	Purpose
Prometheus	Metrics collection
Grafana	Dashboards and visualization
Loki	Centralized logs
OpenTelemetry	Instrumentation
Tempo or Jaeger	Distributed tracing (from Phase 2)
Alertmanager	Alert routing
Host metrics	Server and operating-system health from the Coolify servers
External uptime checks	Availability of the public web and API endpoints from outside the servers

Phase 1 minimum: structured logs, Prometheus metrics, external uptime checks and tested alerts. Dashboards and tracing are added as Coolify services as traffic grows.

11.2 Infrastructure and application metrics

- **Infrastructure:** host CPU and memory, container resource consumption, container restarts, health-check failures, network traffic, disk usage, MinIO capacity, backup execution success.
- **Application:** API request rate, HTTP error rate, response latency, database connection pools, authentication failures, feed response times, notification and email delivery failures, queue depth and job failures.
- **Database:** active connections, slow queries, lock contention, storage growth, backup age and last successful restore test.
- **Membership and governance:** card verification rate and failures, ballot submission rate and errors, rejected ballot attempts, election state changes.

11.3 Proposed service objectives

Indicator	Initial target
API availability	99.5% monthly on the single-server baseline; 99.9% once a standby exists
API latency	p95 below 300 ms for selected endpoints
Error rate	Below 1% under normal operation
Backup recovery	Defined and tested RPO/RTO
Critical alerts	Actionable and tested

These are proposed targets, not guarantees. Validate them through workload testing and operational experience.

12. CI/CD AND DEPLOYMENT ARCHITECTURE

12.1 Pipeline

```
Developer
   |
   v
GitHub Repository
   |
   v
Pull Request
   |
   +----> Unit Tests
   +----> Integration Tests
   +----> Code Quality
   +----> Security Scanning
   +----> OpenAPI Contract Check
   |
   v
Merge to Main
   |
   v
Build and version container images (tag = commit SHA)
   |
   v
Scan and publish images to the registry
   |
   v
Deploy to Staging (Coolify webhook)
   |
   v
Run database migrations
   |
   v
Smoke Tests
   |
   v
Approval / Release Policy
   |
   v
Deploy to Production (rolling update)
   |
   v
Monitor Deployment
   |
   +----> Rollback if necessary
```

12.2 Tools and deployment strategies

- GitHub Actions for CI/CD pipelines; the Go toolchain for reproducible builds.
- Flutter builds for Android and iOS in CI, with signing keys held as secrets, released through the app stores.
- A container registry for versioned images; Coolify for deployment to staging and production.
- Protected branches, automated testing, security scanning and deployment approvals.
- Rolling updates gated by container healthchecks.
- Database migrations are versioned and backward compatible for one release, so a rollback to the previous image remains safe.
- Rollback means redeploying the previous image tag, with a documented procedure that is rehearsed in staging.

13. STORAGE, MEDIA AND EMAIL ARCHITECTURE

A cultural and membership platform will store substantial media and documents over time. Image and video binaries, documents and membership cards must not be stored directly in PostgreSQL.

Data	Storage Engine
User profiles	PostgreSQL
Posts and comments	PostgreSQL
Cultural metadata	PostgreSQL
Membership, governance and audit data	PostgreSQL
Images	MinIO
Documents and membership cards	MinIO
Video	MinIO or external video platform
Search indexes	Search engine
Cached feed data	Redis
Backups	MinIO `backups` bucket and a separate off-site copy

13.1 MinIO buckets

Bucket	Holds	Access
`media-public`	Profile photos, cultural story images, community banners	Public read through the proxy or CDN
`member-private`	Identity documents, application attachments	Presigned URLs only, short expiry
`cards`	Membership card images and PDFs	Presigned URLs only
`documents`	Downloadable forms and organization documents	Per-document visibility rules
`backups`	Database dumps	Never exposed, mirrored off-site

13.2 Media processing workflow

```
User uploads image
       |
       v
Backend validates type and size, issues presigned upload URL
       |
       v
MinIO
       |
       v
Async processing
       |
       +----> Thumbnail generation
       +----> Metadata extraction and removal
       +----> Content moderation
       |
       v
Media published
```

Uploads go directly to MinIO through presigned URLs, which reduces backend bandwidth and improves upload scalability.

13.3 Email delivery over SMTP

- `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD` and `SMTP_FROM` are read from the environment, so changing provider needs no code change.
- Emails are background jobs with retries and backoff. A failed send never fails the user's request.
- Templates live in the repository, with plain-text and HTML versions and one version per supported language.
- SPF, DKIM and DMARC are configured on the sending domain before launch. Bounces and complaints are recorded against the user.
- Staging environments send only to a catch-all inbox.

14. HIGH AVAILABILITY AND DISASTER RECOVERY

14.1 Application availability

- Multiple API instances behind the Coolify proxy.
- CDN-backed web assets with provider redundancy.
- Worker containers with restart policies, and multiple workers where appropriate.
- Proxy health checks, graceful shutdown, rolling updates and automatic container restart.
- A second application server can be added in Coolify to remove the single application server as a failure point.

14.2 Database availability

PostgreSQL runs on a dedicated data server with persistent volumes. The baseline has one primary and no automatic failover. Mitigations are scheduled dumps with off-site copies, continuous WAL archiving for point-in-time recovery as a Phase 1 target, and a standby replica when budget allows. Because the application reads `DATABASE_URL`, a managed PostgreSQL service with multi-AZ failover remains an option that needs a configuration change and no code change.

14.3 Disaster recovery

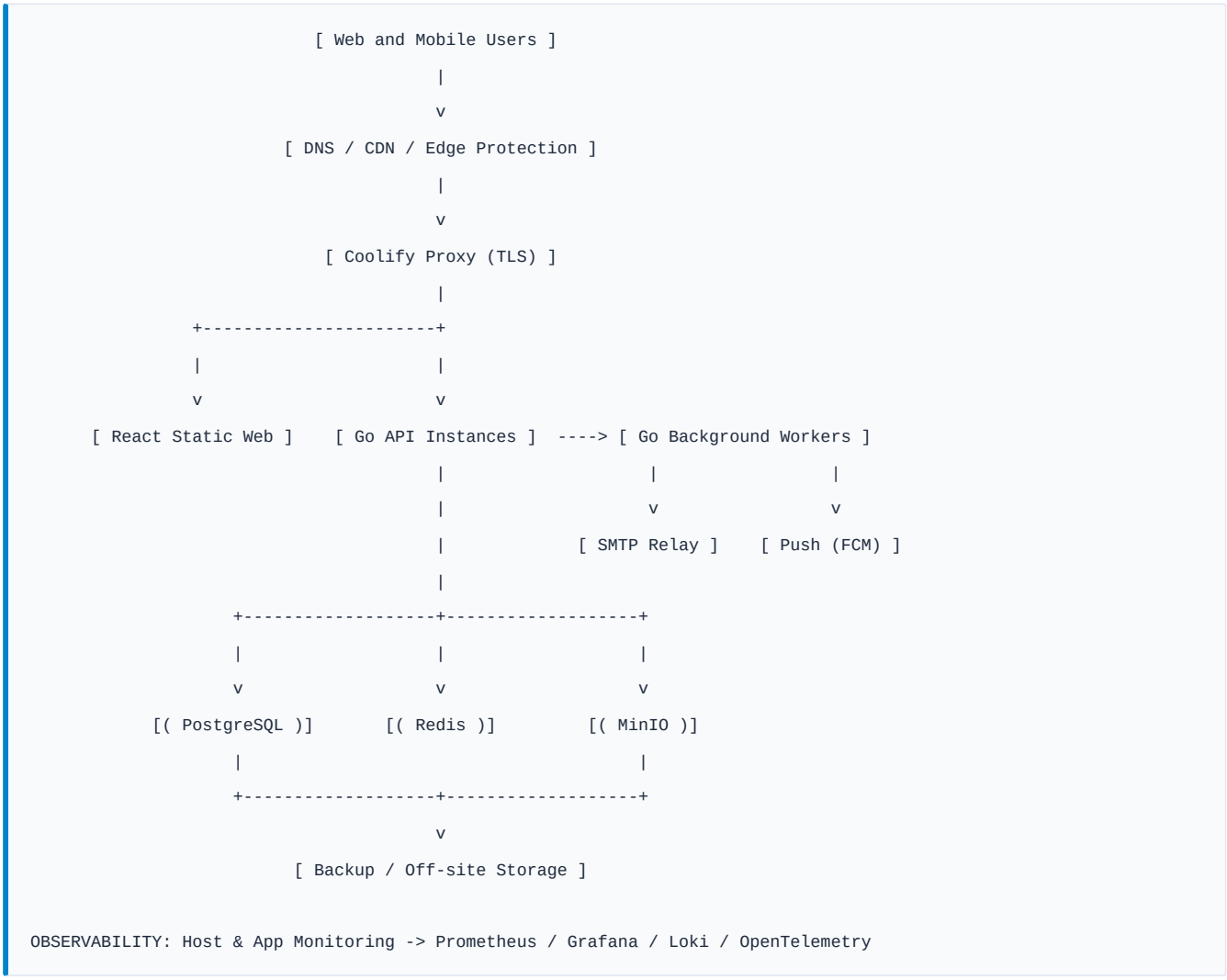
- RPO: define the acceptable data-loss window.
- RTO: define the acceptable restoration time.
- Backup retention, backup encryption and off-site storage with a different provider.
- Database restoration tests and server recreation from version-controlled scripts.
- MinIO data mirrored off-site.
- A full restore drill before launch and every quarter after that.

A backup that has never been restored is not a verified recovery solution.

14.4 Known limits of the Coolify baseline

- Losing the data server stops the platform until it is restored.
- There is no automatic failover between availability zones.
- Database patching, disk capacity and upgrades are operational responsibilities of the team.
- Capacity is added by deliberate action, because Coolify does not scale from metrics.

15. CONSOLIDATED REFERENCE ARCHITECTURE



This is a logical reference architecture. The API and worker containers run on the application server. PostgreSQL, Redis and MinIO run on the private data server. Search starts as PostgreSQL full-text search and moves to a dedicated engine when measurements justify it.

16. ARCHITECTURE DECISIONS

Decision	Proposed choice
Backend	Go (Golang)
Backend pattern	Modular monolith
HTTP framework	Gin
Data access	GORM
Web	React 19, Vite 7, Tailwind CSS, Radix UI, react-router 7
Mobile	Flutter (Dart)
Database	PostgreSQL
Migrations	Versioned SQL migrations (goose or golang-migrate)
Cache and jobs	Redis with Asynq
Search	PostgreSQL full-text search first, dedicated engine on measured need
Media and documents	MinIO (S3-compatible object storage)
Email	SMTP
Push notifications	Firebase Cloud Messaging
API	REST + OpenAPI, with generated TypeScript and Dart clients
Server management	Coolify
Application runtime	Containers managed by Coolify
Initial routing	Coolify proxy
Scaling	Reviewed replica and server additions guided by metrics
CI/CD	GitHub Actions and Coolify
Deployment	Versioned container images with rolling updates
Observability	Prometheus, Grafana, Loki, OpenTelemetry
Production database	PostgreSQL on a dedicated data server, managed service as an option
Governance	Phase 1, symbolic election mode by default
Security	Defense in depth

17. PRODUCTION IMPLEMENTATION AND ROLLOUT ROADMAP

Stage	Build Focus	Production Deliverables
1	Application foundation	Go, React, Flutter, PostgreSQL, Git baseline, OpenAPI contract
2	Servers and networking	Coolify servers, DNS, TLS, proxy, private data network, firewall rules
3	Identity and membership	Identity, member ID, organizations, membership, card and verification
4	Security	Access control policy, MFA, secrets, dependency scanning, API security
5	CI/CD	GitHub Actions, container images, automated deployment to staging and production
6	Community and civic features	Social, communities, culture, events, documents, volunteering, evaluations
7	Governance	Offices, nominations, adoption certificates, elections in symbolic mode, election audit log
8	Notifications and administration	SMTP email, push, moderation, administrative console
9	Observability	Metrics, logs, uptime checks, tested alerts
10	Reliability	Health checks, graceful shutdown, restore drill, failure testing, load tests
11	Production operations	Incident response, SLOs, capacity planning, security review, launch
12	Phase 2	Payments through an aggregator, subscriptions, rewards, meeting rooms, diaspora services, virtual passport
13	Phase 3	Wallet, investment exchange, trade exchange and vending, each after legal review

Production Readiness Gates

All platform services, data pipelines and infrastructure layers must satisfy these gates before serving live traffic:

- **Environment Parity:** Staging and production use the same versioned images and configuration patterns.
- **Zero-Downtime Releases:** Rolling updates backed by health checks and a documented, rehearsed rollback.
- **Reproducible Infrastructure:** Server provisioning, firewall rules and DNS are version-controlled scripts or infrastructure as code.
- **Security Baseline:** Automated source and dependency scanning, least-privilege access, secrets in the secret store, MFA for privileged roles.
- **Resilience Validation:** Container and server failure tests, with a recovery runbook that meets the agreed RTO.
- **Verified Disaster Recovery:** Tested database and file restoration with verified RPO and RTO objectives.
- **Election Integrity:** Ballot secrecy, one-ballot constraints, reproducible results and the audit log tested before any election opens.
- **Deep Observability:** Centralized logging, application metrics and actionable SLO alert thresholds.
- **Performance Standards:** Capacity planning, p95 latency under 300 ms and the agreed availability target.

18. NEXT ARCHITECTURE DELIVERABLE

The next document will be Architecture Blueprint v1.3: Repository, Network Topology & Deployment Design, containing:

1. Complete GitHub repository structure for the API, web, mobile and infrastructure code.
2. Go module and package architecture.
3. PostgreSQL schema ownership and the first migrations.
4. Local Go development workflow and service connections.

5. Coolify server topology and application resource configuration.
6. IP and network segmentation design.
7. Proxy placement and traffic paths.
8. Security policies.
9. Server provisioning scripts and container image design.
10. Development and staging environments.

Implementation will proceed incrementally, testing each component before adding the next.

19. FINAL ARCHITECTURAL POSITION

This architectural blueprint establishes the production baseline for the State 55 Pan-African Digital Community. Every tier, from the React web and Flutter mobile clients to the Go modular monolith in Coolify-managed containers, PostgreSQL transactional storage accessed through GORM, MinIO file storage, SMTP email and end-to-end observability, is designed to meet reliability, security and scalability standards for a phased continental launch.

Validation will be conducted continuously through:

- Functional and integration testing.
- Load testing and security auditing.
- Failure simulation and backup restoration drills.
- Election integrity testing and independent audit before any binding use.
- Deployment rollback and monitoring and incident response exercises.

> **Operational Mandate:** Modular Go services, container-based deployment, automated CI/CD and multi-layered defense-in-depth security. The platform is engineered to deliver a trusted digital home for African culture, heritage, community and member governance, with Phase 2 and Phase 3 capabilities added only after their legal and operational gates are met.